

University of Toronto
Department of Mechanical and Industrial Engineering

Contest 3 Report

Follow Me Robot Companion

MIE443: Mechatronics Systems - Design and Integration
Winter 2023

PRA0102
Thursday Group 7

Adam Cavallo 1004881979

Andres Cervera Rozo 1005469217

Selin Ertan 1004766096

Maggie Vuong 1005473961

Submitted: April 13th, 2023

Table of Contents

1.0 Problem Definition	2
2.0 Strategy	3
2.1 Collaboration	3
3.0 Robot Design and Implementation	5
3.1 Sensory Design	5
3.1.1 RGB Kinect Camera	5
3.1.2 Depth Sensor (IR emitter/receiver)	5
3.1.3 Bumpers	5
3.2. Controller Design	7
3.2.1 contest3.cpp	9
3.2.1.1 Constants	9
FEAR/REVERSE_TIME	9
last_state	9
3.2.1.3 Callback Functions	10
bumperCB	10
followerCB	10
personCB	10
3.2.1.3 User Defined Functions	11
rotate	11
sign	11
3.2.1.4 User Defined States	11
Excited (WorldState = 3)	11
Fear (WorldState = 1)	11
Neutral (WorldState = 0)	12
Rage (WorldState = 4)	12
Sad (WorldState = 2)	12
3.2.1.5 Main Function	13
4.0 Future Improvements	14
5.0 References	15
6.0 Contribution Table	15
Appendix A: contest3.cpp	16

1.0 Problem Definition

For this contest it has a focus on robot emotions and takes place in an empty space with four targets on the ground. Within eight minutes, the robot is required to follow a user to the targets and display four unique emotions in response to four different stimuli. Of the four emotions, two must be primary or reactive while the other two must be secondary or deliberative. These emotions should be multi-modal meaning that they involve multiple forms of expression (i.e. movement, sound, image). Furthermore, two of the stimuli must be defined by the team while the other two are given and are as follows:

1. When the robot loses track of the user it is following
2. When the robot can no longer follow the user due to a short obstacle hitting its bumpers

The robot is to use its sensory inputs to determine the correct response. A summary of the contest's objectives, requirements, and constraints can be seen below in Table 1.1.

Table 1.1: Objectives, Requirements, Constraints of Contest 3

Objectives	• Follow a user • Display 4 unique emotions in response to 4 different stimuli
Requirements	• Development of algorithm using C++ and ROS • Use the Turtlebot follower package • Track the user at a 1m distance • Follow user to 4 different targets • Display 4 unique emotions ○ 2 must be primary/reactive ○ 2 must be secondary/deliberative • Multi-modal display of emotion • React to 4 unique stimuli ○ 2 are given ■ Loses track of the user it is following ■ Cannot follow due an obstacle in its path ○ 2 are user-defined
Constraints	• Time limit of 8 minutes • No human intervention

2.0 Strategy

The main objective of this contest was to connect emotions with an inanimate turtlebot and personify it in such a way that a viewer may see the robot's behaviour and subconsciously believe that it was truly experiencing these emotions. This was achieved through both visual and audible feedback from the robot, as well as through the robot's physical movement, in a way similar to how a human may respond to such stimuli. In order to strengthen the notion of being sentient, a persona was given to the robot through the theme of the film Shrek. This film is quite popular and its vast amount of iconic lines demonstrating strong emotions added further credibility to the robot's emotions. Through sensors and timers, the robot was programmed to both respond to immediate stimuli (primary emotions), and develop a response over time (secondary emotions). Using a state machine, the robot is able to execute the corresponding behaviour for each emotion based on its environment, and then return to a neutral state where it is simply trying to follow the user.

The robot has been programmed to display a set of four emotions: fear (secondary), excited (primary), rage (secondary), and sad (primary). In the event of losing track of the user it is following, the robot triggers the fear emotion, wherein it scans the surroundings in search of the individual. After a designated search period without success, the robot experiences fear, manifested through a backward motion. Upon regaining sight of the user, the robot reacts with excitement, spinning around in elation. Subsequently, when the user walks towards the robots, causing it to go backwards, it experiences anger, culminating in a display of rage by charging towards the individual while emitting a scream. Finally, if the user hits the robot's bumper, it responds with a side-to-side rotational motion indicative of crying.

Using the `sound_play` package, it was possible to integrate an audible response from the robot. This was achieved using a `SoundClient` object that referenced a pre-defined wav file and published the sound to `robotsound`, which is then interpreted and played through the speakers of the laptop. A wav file is preferred in this scenario over the more common MP3 file due to its higher overall sound quality and diminished loss of data. Several different audio clips were used throughout the performance of the contest to demonstrate a range of emotions and to personify the behaviour of the turtlebot.

OpenCV was employed to display the images of emotions, which were initially resized in the Affinity Photoshop software and subsequently exported into PNG files. In the implementation code, corresponding `cv::Mat` files were generated for five emotion states: sad, excited, fear, rage, and neutral. A scaling factor of 1.25 was applied to render the images in full-screen mode. For the primary emotions, the images were displayed immediately following their activation, whereas for secondary emotions, the images were exhibited after a predetermined interval leading to their occurrence.

2.1 Collaboration

Similar to previous contests, our team has chosen to use GitHub in order to collaborate on code. Since there are four emotions and four team members, our

group has decided to split the actions associated with each emotion amongst the team. For the image and sound components, each member was to source their own sounds and images while the two members that did primary emotions implemented the code. The other two members who created the secondary emotions also had the task of creating the state machine. To avoid code conflicts and merge errors, the emotions were originally created as separate functions before being merged in the `contest3.cpp` main function.

3.0 Robot Design and Implementation

3.1 Sensory Design

The sensors used in the algorithm are summarised in the table below.

Table 3.1: Summary of Sensors Used in Algorithm

<u>Sensors</u>	<u>Algorithm Implementation</u>
RGB Kinect Camera	• Follower • Trigger for Fear • Trigger for Excited
Depth sensor	• Follower • Trigger for Fear • Trigger for Excited
Bumpers	• Trigger for sad • Stop condition for rage movement

3.1.1 RGB Kinect Camera

How it works:

- The photodiodes in the RGB camera detect the intensity of light falling on them. Each photodiode is covered with a filter that divides the incoming light into RGB components. The camera's processing algorithms then create a full-color image.

How it is used in our algorithm:

- Use of both colour (RGB) and depth data creates a 3D representation of the environment.
- The fear emotion is triggered when the camera can't detect an object to follow.
- The excited emotion is triggered when the camera detects an object to follow.

3.1.2 Depth Sensor (IR emitter/receiver)

How it works:

- Emits infrared light patterns and then measures the time it takes for the light to bounce back, which allows it to create a depth map of the environment. The depth map is then combined with the colour information from the RGB camera.

How it is used in our algorithm:

- Used to set the variable 'is_alone' as true or false depending on if there is a followable user in view.
- The fear emotion is triggered when the camera can't detect an object to follow.
- The excited emotion is triggered when the camera detects an object to follow.

3.1.3 Bumpers

How it works:

- Utilises a left, right and centre contact sensor at the front of the robot to detect objects that the camera does not see.

How it is used in our algorithm:

- The sad state is triggered when the robot bumps into a person or an object and activates the bumper sensors

3.2. Controller Design

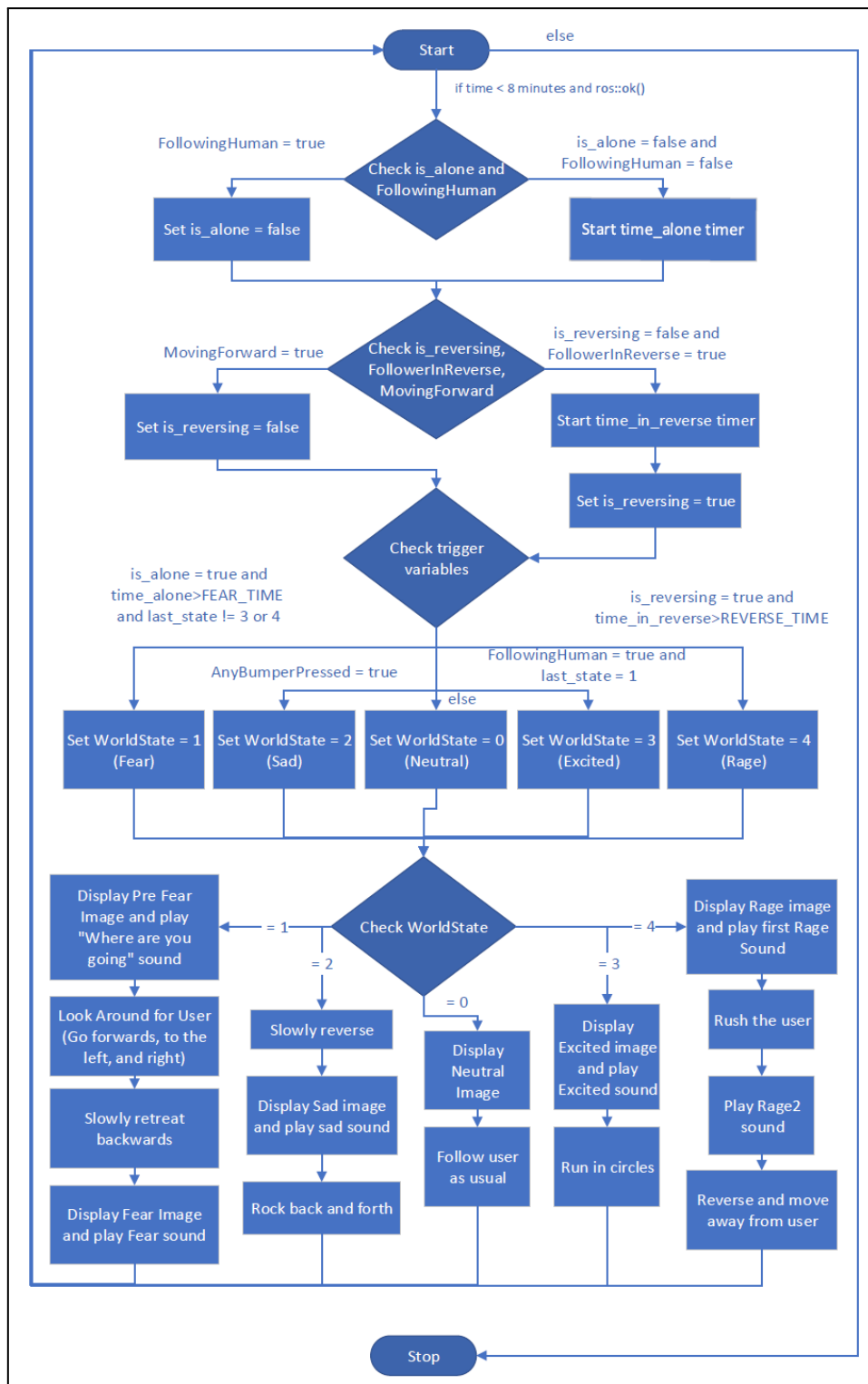


Figure 3.1: Flow Chart of Main While Loop

For this contest, a reactive control architecture was used. This meant that the robot was not planning its movements ahead of time but rather reacting to a given stimulus with a preprogrammed response [1]. In the flow diagram of the main while loop, seen above in Figure 3.1, it can be seen that the robot does not form plans but rather it constantly checks its sensor data in order to act appropriately.

A reactive architecture was sufficient due to its simple environment. Due to the scene being an open space, the robot did not need to sense its environment and plan ahead before following the user. Furthermore, its emotional responses are preprogrammed to different stimuli that are predetermined by the team. Therefore, the robot would not need to “think” or be prepared for what may happen as the team can invoke the desired response at the correct time with the chosen stimulus.

3.2.1 contest3.cpp

The following subsections contain explanations for notable constants, functions, and states. Their implementation can be seen in the contest 3 code which can be found in Appendix A.

3.2.1.1 Constants

Below, these constants were notable or required further explanation beyond their name. An explanation of the constant's function as well as an example of its use will be provided. Other variables can be seen in Appendix A.

FEAR/REVERSE_TIME

- This float represents the chosen time frame for a trigger to occur before the emotional response is evoked.
- For example, for rage, the time required to be in reverse before initiating a response is 3 seconds.

last_state

- This float value is the previous state that the robot was in prior to its current state.
- This was implemented as a condition for entering a state.
- For example, for fear, it can only enter that state if the last_state was not 3 or 4. This means that it cannot be fearful even if it cannot see the user as those states have movement routines that may result in the user not being in the robot's view.

3.2.1.3 Callback Functions

bumperCB

Input:	const kobuki_msgs::BumperEvent::ConstPtr& msg
Output:	N/A
Description:	This callback was taken from contest1.cpp and has the following two functions: 1. Saves all bumper states to the global variable, Bumper. 2. Checks if any bumper is pressed and saves AnyBumperPressed as true if pressed or false if not pressed.

followerCB

Input:	const geometry_msgs::Twist msg
Output:	N/A
Description:	This callback updates the follow_cmd variable. It also sets FollowerInReverse to be false unless the message's linear velocity is less than -0.1, then it will be set to true. Similar to this, it sets MovingForward to true unless the message's linear velocity is less than 0.1, then it will set it to false.

personCB

Input:	const visualization_msgs::Marker msg
Output:	N/A
Description:	It sets the global variables for the followee's x/y/z coordinates. It also sets the FollowingHuman variable to true if the x and y values are non-zero and to false otherwise.

3.2.1.3 User Defined Functions

rotate

Input:	const visualization_msgs::Marker msg
Output:	N/A
Description:	This function was taken from contest1.cpp. It is used to publish commands to rotate the robot based on a desired angle in degrees, inputted as int <i>angle_rot</i>

sign

Input:	const visualization_msgs::Marker msg
Output:	N/A
Description:	This function was taken from contest2.cpp. It returns 1 if the input number is positive and returns -1 if it is negative.

3.2.1.4 User Defined States

Excited (WorldState = 3)

Triggers:	FollowingHuman = true, last_state = 1 (fear)
Stopping Criteria:	Time based
Description:	This is a primary emotion and gets triggered when the robot finds a user to follow after losing the person's sight. As a result, the emotion is executed after the robot is scared when it loses track of the user. When a user is found, the function moves the robot in circles while displaying a happy picture and corresponding audio.

Fear (WorldState = 1)

Triggers:	is_alone = true, time_alone > FEAR_TIME, last_state is not excitement or rage
Stopping Criteria:	Is_alone = false
Description:	This is a secondary emotion that is triggered when the robot loses track of the user it was following. The robot first searches for the user while displaying an audio asking "where did you go?" and displaying an image showing curiosity. After the search,

	when the robot has not found the user, the robot slowly retreats and expresses fear. This is done through a more fearful image and audio display, showing an escalation of emotion. This state is exited after this routine when the user is back in the robot's view and close enough for the robot to follow.
--	---

Neutral (WorldState = 0)

Triggers:	N/A
Stopping Criteria:	Any trigger for the other states.
Description:	When the other emotions aren't triggered, the neutral state of the robot follows a user while displaying a neutral image and no sound.

Rage (WorldState = 4)

Triggers:	Is_reversing = true, time_in_reverse > REVERSE_TIME
Stopping Criteria:	Time based
Description:	Rage is a secondary emotion that gets triggered when the user starts walking towards the robot. As time moving backwards progresses, the robot gets angrier until it reaches the point of rage. It then rushes towards the user and displays a photo until its bumper is hit where it will then play a sound saying "Get out of my Swamp!". Then it will reverse and angrily turn away from the user while roaring.

Sad (WorldState = 2)

Triggers:	AnyBumperPressed = true
Stopping Criteria:	Timer based
Description:	This is a primary emotion that gets triggered when the robot's bumpers are hit. The robot mimics rocking back and forth by rotating side to side while displaying a sad image and audio. Note: This emotion doesn't get triggered when the bumpers are hit during the rage emotion because the bumper callback is not triggered until after the rage emotion has come to an end.

3.2.1.5 Main Function

A step by step explanation of the contest file's main function can be seen below while its code can be found in Appendix A.

Main function - Step through explanation

Setup:

1. Node gets initialised as 'emotional_follower' with frequency of 10Hz
2. Initialize Sound_play node
3. Set path to folder of sounds
4. Establish publisher to teleop topic
5. Establish subscriber to follower topic
6. Establish subscriber to bumper topic
7. Establish subscriber to person topic
8. Initialize the timers
9. Initialize images as OpenCV objects
10. Resize images
11. Initialize Image transporter
12. Start the timers
13. Initialize variables (and WorldState)

While under 8 mins:

1. SpinOnce()
2. Determine if the user is out of sight, and start a timer if so
3. Determine if the robot is reversing, and start another timer if so
4. If the robot has been alone for sufficient time, set the fear emotion
5. If a bumper has been pressed, set the sad emotion
6. If the last emotion was fear and the user is visible again, set the excited emotion
7. If the robot has been reversing for sufficient time, set the rage emotion
8. Complete the action that goes with the determined emotion use a switch-case block
9. Return to beginning of while loop

4.0 Future Improvements

To further improve the emotional expression of our program, several modifications can be made.

For instance, to make our emotions more explicit we could implement dynamic visuals such as a video or a set of changing images to display a single emotion. Playing a video is similar to displaying a picture and can be done using OpenCV. The additional commands are as follows: `cap = cv.VideoCapture('filename.avi')`, `video = cap.read()`, `cv.imshow('video')`. This could allow for better representation of the emotions and could complement the audio that is being played.

Furthermore, the Team could make the emotions contextual by changing certain aspects of the emotions based on different scenarios that trigger it. Multiple triggers could be used for the same emotion type, but different audio clips or images could be played or displayed that relate more towards the scenario. For example, rage can result from different triggers such as forcing the robot to reverse, or standing still for too long. The associated audio clips can be different for these two scenarios to express the trigger for the emotion more clearly. An impatient robot could warn that it is going to react with rage by first announcing that it is getting bored due to lack of activity. Similarly, forcing a robot to reverse can first express that its personal space is being violated. This can be further improved by implementing a mood bar visual to demonstrate how certain actions affect the mood of the robot and lead to a certain reaction. Overall, having these additional expressions can help the audience to better determine the trigger event and its type (time based or repetition based).

Lastly, additional methods of emotion expression can be used other than pictures/videos, sound and movement. For instance, using arduino an external LED can be implemented to display different colours based on the emotion. A colour of green could represent positive emotions, while a colour of red or flashing LED could be interpreted as a negative emotion or a warning for an upcoming negative action.

5.0 References

- [1] G. Nejat, "MIE 443 LECTURE 4: Intelligent Control," *Quercus*. [Online]. Available: <https://q.utoronto.ca/courses/288207/files/folder/Contests/Contest%20%231?preview=24256737>. [Accessed: 16-Feb-2023].

6.0 Contribution Table

Adam Cavallo	Developed the behaviour for sad emotion
Andres Cervera Rozo	Developed the behaviour for rage emotion
Selin Ertan	Developed the behaviour for excited emotion
Maggie Vuong	Developed the behaviour for fear emotion

Appendix A: contest3.cpp

```
#include <header.h>
#include <ros/package.h>
#include <imageTransporter.hpp>
#include <chrono>
#include <kobuki_msgs/BumperEvent.h>
#include <visualization_msgs/Marker.h>
#include <opencv2/core.hpp>
#include <cv.h>
#include <cv_bridge/cv_bridge.h>

// #include "led_manager/LedAnim.h" // Selin

using namespace std;
#define LOOP_RATE 10 // Rate for while loops that dictate
// callback and publish frequency (Hz)
#define FORWARD_DIST 0.5
#define TURNING_V 0.6 //Adam
#define BACKWARD_V -0.3 //Adam
#define N BUMPER (3) // Number of bumpers on kobuki base
#define RAGE_VEL 0.5
#define RAGE_TURN 1.5
#define REVERSE_TIME 3
#define BACKWARD_T 0.5
#define FEAR_TIME 2.5
#define DEG2RAD(deg) ((deg) * M_PI / 180.) // Inverse conversion function

// sound_play::SoundClient sc;
geometry_msgs::Twist follow_cmd;
geometry_msgs::Twist Vel; // Create
// message for velocities as Twist type
ros::Publisher VelPub;

// std::string path_to_sounds
// ("/home/thursday/catkin_ws/src/MIE443-Turtlebot-Rover/mie443_contest3/sounds");
string path_to_sounds = ros::package::getPath("mie443_contest3") + "/sounds/";

uint8_t Bumper[3] = {kobuki_msgs::BumperEvent::RELEASED,
kobuki_msgs::BumperEvent::RELEASED, kobuki_msgs::BumperEvent::RELEASED};
bool AnyBumperPressed = false; // Reset variable to false
bool FollowerInReverse = false;
bool MovingForward = true;

bool FollowingHuman = true;

int WorldState;

int sign(float number) {
    return (number>0)? 1: -1; // Positive -> 1, otherwise
-1
}

void personCB (const visualization_msgs::Marker msg) {
```

```

        float person_x = msg.pose.position.x;
        float person_y = msg.pose.position.y;
        float person_z = msg.pose.position.z;
        // ROS_INFO("Person x: %f, Person y: %f, Person z: %f", person_x, person_y,
person_z);
        if ((person_x != 0) && (person_y != 0)) FollowingHuman = true;
        else FollowingHuman = false;
    }

void followerCB(const geometry_msgs::Twist msg){ // Andres
    follow_cmd = msg;
    FollowerInReverse = false;
    MovingForward = true;
    if (msg.linear.x < -0.1) FollowerInReverse = true;
    else FollowerInReverse = false;
    if (msg.linear.x > 0.1) MovingForward = true;
    else MovingForward = false;
}

//Adam's branch////////////////////////////////////
void bumperCB(const kobuki_msgs::BumperEvent::ConstPtr& msg){
    // Access using bumper[kobuki_msgs::BumperEvent::{}] LEFT, CENTER, or RIGHT
    Bumper[msg->bumper] = msg->state; // Assigns bumper array to match
message based on msg message
    AnyBumperPressed = false;
    for (uint8_t i = 0; i < N BUMPER; i++){
        if (Bumper[i] == kobuki_msgs::BumperEvent::PRESSED) AnyBumperPressed = true; //
bumper[0] = leftState, bumper[1] = centerState, bumper[2] = rightState
    }
    // if (AnyBumperPressed) WorldState = 2; Moved to main
}

void rotate(ros::Publisher vel_pub, int angle_rot){ // Called with
input as an angle in degree
    ROS_INFO("rotating by: %i",angle_rot);

    int dir = sign(angle_rot);
    // ROS_INFO("direction: %i", dir);
    float angle_rad = DEG2RAD(dir*angle_rot); // Function takes in
an angle in degrees, converts it to rad
    float turning_time = angle_rad/TURNING_V; // Calculates the
time needed to turn based on constant turning speed, and the angle

    std::chrono::time_point<std::chrono::system_clock> start; // Initialize timer
    start = std::chrono::system_clock::now(); // Start new timer
at current time
    uint64_t seconds_elapsed = 0; // New variable for
time that has passed

    ros::Rate loop_rate(LOOP_RATE);
    Vel.linear.x = 0;
    Vel.angular.z = dir * TURNING_V; // Set turning speed
    // ROS_INFO("direction: %f", dir*TURNING_V);
    while (ros::ok() && seconds_elapsed <= turning_time) { // Turn until the
turning time needed to complete the angle is passed
        vel_pub.publish(Vel); // Start turning
    }
}

```

```

        loop_rate.sleep();
        seconds_elapsed =
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-start)
.count(); //count how much time has passed
    }
}
// -----
int main(int argc, char **argv)
{
    ros::init(argc, argv, "emotional_follower");
    ros::NodeHandle nh;

    sound_play::SoundClient sc;
    // string path_to_sounds = ros::package::getPath("mie443_contest3") + "/sounds/";
    // string path_to_sounds = ros::package::getPath("mie443_contest3") + "/sounds/";
    teleController eStop;

    //publishers
    VelPub = nh.advertise<geometry_msgs::Twist>("cmd_vel_mux/input/teleop",1);
    //ros::ServiceClient led_client =
    nh.serviceClient<led_manager::LedAnim>("led_anim");

    //subscribers
    ros::Subscriber follower =
nh.subscribe("follower_velocity_smoother/smooth_cmd_vel", 10, &followerCB);
    ros::Subscriber bumper = nh.subscribe("mobile_base/events/bumper", 10,
&bumperCB);
    ros::Subscriber person = nh.subscribe("turtlebot_follower/marker", 10,
&personCB);

    // contest count down timer
    ros::Rate loop_rate(10);
    std::chrono::time_point<std::chrono::system_clock> start;
    std::chrono::time_point<std::chrono::system_clock> reverse_start;
    std::chrono::time_point<std::chrono::system_clock> fear_start;

    // creating the image files for emotions
    cv::Mat sad_image =
cv::imread("/home/thursday/catkin_ws/src/MIE443-Turtlebot-Rover/mie443_contest3/images/S
ad.png",cv::IMREAD_UNCHANGED);
    cv::Mat excited_image =
cv::imread("/home/thursday/catkin_ws/src/MIE443-Turtlebot-Rover/mie443_contest3/images/E
xcited.png",cv::IMREAD_UNCHANGED);
    cv::Mat fear_image =
cv::imread("/home/thursday/catkin_ws/src/MIE443-Turtlebot-Rover/mie443_contest3/images/F
ear.png",cv::IMREAD_UNCHANGED);
    cv::Mat rage_image =
cv::imread("/home/thursday/catkin_ws/src/MIE443-Turtlebot-Rover/mie443_contest3/images/R
age.png",cv::IMREAD_UNCHANGED);
    cv::Mat neutral_image =
cv::imread("/home/thursday/catkin_ws/src/MIE443-Turtlebot-Rover/mie443_contest3/images/N
eutral.png",cv::IMREAD_UNCHANGED);
    cv::Mat curious_image =
cv::imread("/home/thursday/catkin_ws/src/MIE443-Turtlebot-Rover/mie443_contest3/images/C
urious.png",cv::IMREAD_UNCHANGED);

```

```

double scaleFactor = 1.25;

cv::resize(sad_image, sad_image, cv::Size(sad_image.cols*scaleFactor,
sad_image.rows*scaleFactor), cv::INTER_LINEAR);
cv::resize(excited_image, excited_image, cv::Size(excited_image.cols*scaleFactor,
excited_image.rows*scaleFactor), cv::INTER_LINEAR);
cv::resize(fear_image, fear_image, cv::Size(fear_image.cols*scaleFactor,
fear_image.rows*scaleFactor), cv::INTER_LINEAR);
cv::resize(rage_image, rage_image, cv::Size(rage_image.cols*scaleFactor,
rage_image.rows*scaleFactor), cv::INTER_LINEAR);
cv::resize(neutral_image, neutral_image, cv::Size(neutral_image.cols*scaleFactor,
neutral_image.rows*scaleFactor), cv::INTER_LINEAR);
cv::resize(curious_image, curious_image, cv::Size(curious_image.cols*scaleFactor,
curious_image.rows*scaleFactor), cv::INTER_LINEAR);

imageTransporter rgbTransport("camera/image/",
sensor_msgs::image_encodings::BGR8); //--for Webcam
//imageTransporter rgbTransport("camera/rgb/image_raw",
sensor_msgs::image_encodings::BGR8); //--for turtlebot Camera
imageTransporter depthTransport("camera/depth_registered/image_raw",
sensor_msgs::image_encodings::TYPE_32FC1);

// geometry_msgs::Twist vel; //unnecessary, likely will be deleted
// vel.angular.z = 0.2;
// vel.linear.x = 0.0;

start = std::chrono::system_clock::now();
reverse_start = start;
fear_start = start;
// buffer_start = std::chrono::system_clock::now(); May cause issues
// reverse_start = std::chrono::system_clock::now();

WorldState = 0;
uint64_t seconds_elapsed = 0;
uint64_t time_in_reverse = 0;
uint64_t time_alone = 0;
uint8_t last_state = 0;
bool is_alone = false;
bool is_reversing = false;
ros::spinOnce();
ros::Duration(0.5).sleep();
ROS_INFO("Entering while loop");
while(ros::ok() && seconds_elapsed <= 480){
    ros::spinOnce();
    ROS_INFO("FollowingHuman: %i, AnyBumperPressed: %i, MovingForward: %i,
FollowerInReverse: %i", FollowingHuman, AnyBumperPressed, MovingForward,
FollowerInReverse);

// // try {
// //     if(isValid) {
// //         img.release(); //previous image is released from memory
// //     }
// //     img = (cv_bridge::toCvShare(msg, IMAGE_TYPE)->image).clone();
// //     isValid = true;
// // } catch (cv_bridge::Exception& e) {

```

```

// //      std::cout << "ERROR: Could not convert from " << msg->encoding.c_str()
// //      << " to " << IMAGE_TYPE.c_str() << "!" << std::endl;
// //      isValid = false;
// //  }
// // }

        time_in_reverse =
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-reverse_start).count();
        time_alone =
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-fear_start).count();

        if (!is_alone && !FollowingHuman){
            time_in_reverse = 0;
            is_reversing = false;
            fear_start = std::chrono::system_clock::now();
            time_alone =
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-fear_start).count();
            is_alone = true;
        }
        else if (FollowingHuman) is_alone = false;

        if (!is_reversing && FollowerInReverse){
            reverse_start = std::chrono::system_clock::now();
            time_in_reverse =
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-reverse_start).count();
            is_reversing = true;
        }
        else if (MovingForward) is_reversing = false;
        ROS_INFO("last_state: %i, time: %i, reverse time: %i, alone time: %i, reversing?: %i, alone?: %i", last_state, seconds_elapsed, time_in_reverse, time_alone, is_reversing, is_alone);
        // Fear Mode Conditions
        if ((is_alone) && (time_alone > FEAR_TIME) && (last_state != 3) && (last_state != 4)) WorldState = 1;

        // Sad Mode Conditions
        else if (AnyBumperPressed) WorldState = 2;

        // Excitement Mode Conditions
        else if ((FollowingHuman) && (last_state == 1)) WorldState = 3;

        // Rage Mode Conditions
        else if (is_reversing && (time_in_reverse > REVERSE_TIME)) WorldState = 4;

        else WorldState = 0;

        ROS_INFO("Worldstate: ", WorldState);
        switch (WorldState) {
            case 0: {
                // neutralMode(VelPub); //idk play music or show images or

```

```

something
    cv::imshow("view",neutral_image);
    cv::waitKey(1);
    VelPub.publish(follow_cmd);
    ROS_INFO("Following Human as normal");
    break; }
case 1: {
    ROS_INFO("Entering fear mode");

    float fear_vel = 0.4;
    float fear_rev_vel = -0.1;
    float fear_rot_vel = 2.0;
    int fear_forward_time = 0.6;
    int fear_rev_time = 5;
    bool spun = false;
    // ros::Rate loop_rate(LOOP_RATE);
    std::chrono::time_point<std::chrono::system_clock>
fear_start;

    uint64_t fear_seconds_elapsed = 0;

    Vel.angular.z = 0.0;
    Vel.linear.x = 0.0;
//
    while (ros::ok() && !AnyBumperPressed && !FollowingHuman){
        // spin around
        cv::imshow("view",curious_image);
        while (!spun){
            cv::waitKey(1);
            Vel.linear.x = 0.0;
            Vel.angular.z = fear_rot_vel;
            VelPub.publish(Vel);
            sc.playWave(path_to_sounds + "Fear_1.wav");
            rotate(VelPub, 180);
            rotate(VelPub, 180);
            spun = true;
        }

        // go forwards
        fear_start = std::chrono::system_clock::now();
        ROS_INFO("Fear Forward");
        cv::imshow("view",curious_image);
        while (ros::ok() &&
fear_seconds_elapsed<=fear_forward_time){
            ros::spinOnce();
            cv::waitKey(1);
            Vel.linear.x = fear_vel;
            Vel.angular.z = 0.0;
            VelPub.publish(Vel);
            loop_rate.sleep();
            fear_seconds_elapsed =
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-fear_s
tart).count();
        }
//
        // turn slgihtly and go forwards
        Vel.linear.x = 0.0;

```

```

        Vel.angular.z = fear_rot_vel;
        VelPub.publish(Vel);
        rotate(VelPub, 90);
        fear_start = std::chrono::system_clock::now();
        fear_seconds_elapsed = 0;
        ROS_INFO("Turn slightly and forward");
        cv::imshow("view", curious_image);
        while (ros::ok() &&
fear_seconds_elapsed<=fear_forward_time){
            ros::spinOnce();
            cv::waitKey(1);
            ROS_INFO("2nd forward");
            Vel.linear.x = fear_vel;
            Vel.angular.z = 0.0;
            VelPub.publish(Vel);
            loop_rate.sleep();
            fear_seconds_elapsed =
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-fear_s
tart).count();
        }
        //
        // turn slightly in other direction and go forwards

        Vel.linear.x = 0.0;
        Vel.angular.z = fear_rot_vel;
        VelPub.publish(Vel);
        rotate(VelPub, -180);
        fear_start = std::chrono::system_clock::now();
        fear_seconds_elapsed = 0;
        ROS_INFO("Turn slightly other dir. and forward");
        cv::imshow("view", curious_image);
        while (ros::ok() &&
fear_seconds_elapsed<=fear_forward_time){
            ros::spinOnce();
            cv::waitKey(1);
            Vel.linear.x = fear_vel;
            Vel.angular.z = 0.0;
            VelPub.publish(Vel);
            loop_rate.sleep();
            fear_seconds_elapsed =
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-fear_s
tart).count();
        }
        //
        // slowly go backwards and express fear sound
        Vel.linear.x = 0.0;
        Vel.angular.z = 0.2;
        VelPub.publish(Vel);
        rotate(VelPub, 90);
        fear_start = std::chrono::system_clock::now();
        ROS_INFO("backwards and noise");
        cv::imshow("view", fear_image);
        fear_seconds_elapsed = 0;
        sc.playWave(path_to_sounds + "Fear_2.wav");
        while (ros::ok() &&
fear_seconds_elapsed<=fear_rev_time){

```

```

// sc.playWave(path_to_sounds +
"Fear_3.wav");

ROS_INFO("BACKWARDS");
Vel.linear.x = fear_rev_vel;
Vel.angular.z = 0.0;
VelPub.publish(Vel);
ros::spinOnce();
cv::waitKey(1);
loop_rate.sleep();
fear_seconds_elapsed =

std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-fear_s
tart).count();

}
cv::destroyAllWindows();
break;
//sc.stopPlayingWave(); //stops the sound CAUSES
CATKIN_MAKE_ERROR

}
time_in_reverse = 0;
is_reversing = false;

//
//

// fearMode(VelPub);
ROS_INFO("Broke fear mode");
last_state = 1;
is_alone = false;
cv::destroyAllWindows();
break;}

case 2:{
ROS_INFO("Hit something, entering sad mode");
// sadMode(VelPub);
cv::imshow("view",sad_image);
Vel.linear.x = BACKWARD_V;

// Set linear to Twist
Vel.angular.z = 0;
uint64_t sad_seconds_elapsed = 0;
// New variable for time that has passed
float backward_time = 0.25/abs(BACKWARD_V); //0.5 m
backwards

// ros::Rate loop_rate(LOOP_RATE);
std::chrono::time_point<std::chrono::system_clock>
sad_start;

sad_start = std::chrono::system_clock::now();
// sad_Start new timer at current time
ROS_INFO("Going backwards because sad");
while (ros::ok() && sad_seconds_elapsed <= backward_time) {
// Kobuki diameter is 0.3515m, we want to travel half (3.5s x 0.05m/s)
VelPub.publish(Vel);
// Publish cmd_vel to teleop mux input
loop_rate.sleep();
sad_seconds_elapsed =

std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-sad_st
art).count(); // Count how much time has passed

//ROS_INFO("sad_seconds_elapsed:%i,backward_T:%f",sad_seconds_elapsed,BACKWARD_T);

```



```

    }
    //sad_start being sad

    ros::Time start_time0 = ros::Time::now();
    sc.playWave(path_to_sounds + "Sad.wav"); // starts playing
crying noises
    while ((ros::Time::now()-start_time0).toSec() < 11.0){
        // cv::imshow("view",sad_image);
        cv::waitKey(1);
        rotate(VelPub, -15); //rotate back an fourth while
crying
        //delay()
        rotate(VelPub,30);
        //delay()
        rotate(VelPub,-30);
        //delay()
        rotate(VelPub,30);
        //delay()
        rotate(VelPub,-30);
        //delay()
        rotate(VelPub, 15);
        //delay
    }
    WorldState = 0;
    //sc.stopPlayingWave(); //stops the sound CAUSES
CATKIN_MAKE ERROR

    ROS_INFO("Broke sad mode");
    last_state = 2;
    cv::destroyAllWindows();
    break;
case 3:{
    sc.playWave(path_to_sounds + "Excited.wav");
    ROS_INFO("Found my person, getting excited");
    // excitedMode(VelPub);

    // Move TurtleBot in circles for 10 seconds
    Vel.angular.z = 1.5;
    Vel.linear.x = 0.4;
// Set Linear to Twist
    uint64_t excited_seconds_elapsed = 0;
// New variable for time that has passed
    float move_time = 10; //0.5 m backwards
    // ros::Rate loop_rate(LOOP_RATE);
    std::chrono::time_point<std::chrono::system_clock>
excited_start;
    excited_start = std::chrono::system_clock::now();
// excited_Start new timer at current time
    cv::imshow("view", excited_image);
    while (ros::ok() && excited_seconds_elapsed <= move_time) {
// Kobuki diameter is 0.3515m, we want to travel half (3.5s x 0.05m/s)
        cv::waitKey(1);
        VelPub.publish(Vel);
// Publish cmd_vel to teleop mux input

```

```

        loop_rate.sleep();
        excited_seconds_elapsed =
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-excite
d_start).count(); // Count how much time has passed

//ROS_INFO("excited_seconds_elapsed:%i,backward_T:%f",excited_seconds_elapsed,BACKWARD_T
);

    }

    // Play happy/exciting song for 3 seconds
    // sc.playWave(path_to_sounds + "sound.wav"); //add
happy/exciting sound and modify the name
    // sound_play::SoundClient sc;

    // // Blink the LEDs in different colors
    // led_manager::LedAnim led_anim;
    // led_anim.request.name = "blink";
    // led_anim.request.color.r = 255;
    // led_anim.request.color.g = 0;
    // led_anim.request.color.b = 0;
    // led_anim.request.frequency = 5;
    // led_anim.request.duration = 5.0;
    // led_client.call(led_anim);
    WorldState = 0;

    ROS_INFO("Breaking excited mode");
    last_state = 3;
    cv::destroyAllWindows();
    break;}

case 4:{
    cv::destroyAllWindows();
    ROS_INFO("Got intimidated, getting angry");
    // rageMode(VelPub);

    // Shake a little
    // Publish forward in a loop until bumper is pressed
    // Make noises
    // Turn screen red

    ros::Duration(0.5).sleep();

    // ros::Rate loop_rate(LOOP_RATE);
    std::chrono::time_point<std::chrono::system_clock>
rage_start; // Initialize timer

    Vel.angular.z = 0.0;
    Vel.linear.x = RAGE_VEL;
    uint64_t rage_seconds_elapsed = 0;
    // Variable for time that has passed

    ROS_INFO("Rushing person");
    cv::imshow("view", rage_image);
    while (ros::ok() && !AnyBumperPressed) {
        // Move forward until it hits feet
        cv::waitKey(1);

```

```

        VelPub.publish(Vel);
        ros::spinOnce();
// Listen to all subscriptions once
        loop_rate.sleep();
    }
    // "get out of my way"
    sc.playWave(path_to_sounds + "Rage_1.wav"); //Move/get out,
consider putting in bumper callback

    ROS_INFO("Foot hit, reversing");

    Vel.linear.x = BACKWARD_V;
// Set linear to Twist
    Vel.angular.z = 0;
    rage_seconds_elapsed = 0;
// New variable for time that has passed
    int angle = 0;

    rage_start = std::chrono::system_clock::now();
// rage_Start new timer at current time
    // Move back a little
    while (ros::ok() && rage_seconds_elapsed <= BACKWARD_T) {
// Kobuki diameter is 0.3515m, we want to travel half (3.5s x 0.05m/s)
        VelPub.publish(Vel);
// Publish cmd_vel to teleop mux input
        loop_rate.sleep();
        rage_seconds_elapsed =
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-rage_s
tart).count(); // Count how much time has passed

//ROS_INFO("rage_seconds_elapsed:%i,backward_T:%f",rage_seconds_elapsed,BACKWARD_T);
    }
    // Turn right
    rotate(VelPub, -90);

    Vel.angular.z = 0.0;
    Vel.linear.x = RAGE_VEL;
    rage_seconds_elapsed = 0;
// Variable for time that has passed
    float time_forward = abs(FORWARD_DIST/RAGE_VEL);

    rage_start = std::chrono::system_clock::now();
// rage_Start the timer

    ros::spinOnce();
    ROS_INFO("Move away from victim");
    while (ros::ok() && rage_seconds_elapsed <= time_forward &&
!AnyBumperPressed) { //Move forward a few steps
        VelPub.publish(Vel);
        ros::spinOnce();
// Listen to all subscriptions once
        loop_rate.sleep();
        rage_seconds_elapsed =
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-rage_s
tart).count(); //count how much time has passed
    }

```

```

sc.playWave(path_to_sounds + "Rage_2.wav");
//Roooooooooooooarrrrr

Vel.angular.z = RAGE_TURN;
Vel.linear.x = 0.0;
rage_seconds_elapsed = 0;
// Variable for time that has passed
float time_turning = 6.0;

rage_start = std::chrono::system_clock::now();
// rage_Start the timer

ros::spinOnce();
ROS_INFO("Angry spin");
while (ros::ok() && rage_seconds_elapsed <= time_turning) {
// Spin for 3 seconds

    VelPub.publish(Vel);
    ros::spinOnce();
// Listen to all subscriptions once

    loop_rate.sleep();
    rage_seconds_elapsed =
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-rage_s
tart).count(); //count how much time has passed
}

    ROS_INFO("Breaking rage mode");
    is_reversing = false;
    last_state = 4;
    cv::destroyAllWindows();
    break;}}

}

seconds_elapsed =
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-start)
.count();

    loop_rate.sleep();
}
ROS_INFO("Main while loop ended");
//Play "I'm a Believer" and flash the lights

return 0;
}

```